VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY



FACULTY OF INFORMATION TECHNOLOGY

LAB REPORT

Lab 02: Xv6 System Call

Instructor:	Msc. Viet-Long Le	
Leader:	Phuc-Khang Nguyen	24120068
Members:	Minh-Trong Pham	24120151
	Trong-Nghia Hoang	24120103

Ho Chi Minh City, Ngày 4 tháng 11 năm 2026

Table of Contents

1 trace.c **1**

1.1 Triển khai 1

1.2 Kiểm thử 2

2 sysinfo.c **5**

2.1 Triển khai 5

2.2 Kiểm thử 6

3 pstree.c **8**

3.1 Triển khai 8

3.2 Kiểm thử 10

4 Bảng phân công nhiệm vụ **12**

1 trace.c

Trong hệ điều hành xv6, mỗi lời gọi hệ thống hay còn được gọi là system call thì đều được định danh duy nhất bởi một số nguyên dương, bắt đầu từ `SYS_fork = 1` đến `SYS_close = 21` trong tệp tiêu đề `kernel/syscall.h`, đại diện cho 21 system call cơ bản nhất được định nghĩa [1, 2]. Để tích hợp tính năng theo dõi tiến trình hay còn gọi là `trace`, ta cần thiết lập thêm mã định danh `SYS_trace = 22`. Số hiệu này đóng một vai trò kép trong kiến trúc hệ thống: vừa là chỉ số tra cứu con trỏ hàm xử lý trong bảng điều phối `syscalls[]`, vừa xác định vị trí bit thứ N (`1 « N`) trong mặt nạ theo dõi (`trace_mask`). Nhờ cơ chế bitmask, chương trình người dùng có thể linh hoạt giám sát một hoặc nhiều syscall đồng thời chỉ thông qua một số nguyên duy nhất, đồng thời cho phép kernel kiểm tra nhanh điều kiện kích hoạt thông qua phép toán bitwise.

1.1 Triển khai

Để triển khai được tính năng `trace`, nhóm cần tạo được một file `trace.c` ở trong thư mục `user/`. Đây sẽ là nơi nhận lệnh `trace <mask <command` để thực hiện quá trình theo dõi. Vì mỗi lệnh `trace` đều phải nhận vào một mặt nạ, đại diện cho các system call đang được theo dõi tại một thời điểm t nào đó. Do đó, ta phải định nghĩa một số nguyên để có thể lưu trữ điều này ở ngay trong chính `struct proc` tại `kernel/proc.h`. Và khi user gọi `trace` thì `mask` sẽ được truyền như là số nguyên vào system call `trace(int)`. Từ đó mà ở phía dưới, kernel sẽ dùng hàm `sys_trace()` (do nhóm tự khai báo) tại `kernel/sysproc.c` sử dụng `argint(0, &mask)` để tiếp nhận tham số này và lưu nó trực tiếp vào tiến trình hiện hành (`myproc()->trace_mask`).

Quá trình xử lý đơn giản chỉ cần ta chuyển đổi số `<mask` thành một số nguyên và truyền vào hàm `trace(int)` để thực hiện việc theo dõi. Tuy nhiên, ở trong đề bài có một yêu cầu đó là `trace` sẽ bật theo dõi cho tiến trình gọi nó và bất kỳ tiến trình con nào mà nó phân nhánh sau đó, nhưng không được ảnh hưởng đến các quy trình khác [1]. Do đó, khi tạo ra một proc mới ở trong hàm `allocproc(void)` thì ta cần đặt lại về 0 để biểu thị cho việc là `proc` hiện tại chưa được theo dõi. Tiếp theo đó là khi một tiến trình gọi tiến trình con thì chính tiến trình con phải được kế thừa toàn bộ thông tin về các system call đang theo dõi từ cha. Như vậy, ta cần sao chép `trace_mask` của cha và gán nó vào cho con. Nhờ đó, toàn bộ các tiến trình con cháu phân nhánh sau này đều được theo dõi liên tục mà vẫn bảo đảm tính cô lập, không làm ảnh hưởng đến các tiến trình độc lập khác trong hệ thống.

Bắt và In : Sau khi mà ta đã triển khai xong được cơ chế lưu trữ và kế thừa thì bước quan trọng tiếp theo đó chính là tiến hành bắt được các thông tin của syscall và in nó ra màn hình hệ thống tại hàm điều phối `syscall(void)` trong file `kernel/syscall.c`. Vì mặc định trong hệ điều hành `xv6` thì các syscall chỉ được lưu dưới dạng các con số, nhưng bài toán yêu cầu ta phải in ra màn hình được tên của syscall, nên để thuận tiện cho việc truy xuất và tiết kiệm thời gian, thì nhóm khai báo một mảng chuỗi ký tự `syscall_names[]` để có thể in tên ra màn hình một cách nhanh chóng. Và mỗi khi một system call được gọi, thì sau khi mà **con trỏ hàm** tương ứng đã thực thi xong thì trả về kết quả lên thanh ghi `p->trapframe->a0`, và từ đó thì hệ thống sẽ tiến hành kiểm tra xem bit thứ `num` của system call đó có đang được bật trong `trace_mask` hay không. Nếu bật thì kernel sẽ in ra màn hình thông tin truy vết tương ứng theo đề bài yêu cầu đó là: `<pid>: syscall <name> -> <return_val>`.

1.2 Kiểm thử

Sau khi hoàn tất quá trình triển khai và xử lý lời gọi hệ thống `trace`, nhóm tiến hành xây dựng các kịch bản kiểm thử nhằm kiểm tra tính đúng đắn của giải pháp trên các trường hợp thông thường và trường hợp biên. Bộ kiểm thử bao phủ từ các chức năng cơ bản, trường hợp biên, tính kế thừa phân nhánh cho đến khả năng xử lý lỗi hệ thống và tham số đầu vào (Bảng 1).

Lệnh kiểm thử	**Mục đích kiểm tra**
`trace 32 grep hello README`	Theo dõi duy nhất một system call (`SYS_read = 5`, mặt nạ $1 \ll 5 = 32$). Hệ thống chỉ in các lời gọi `read`.
`trace 32896 grep hello README`	Theo dõi kết hợp `SYS_exec` (7) và `SYS_open` (15) với mặt nạ $(1 \ll 7) \mid (1 \ll 15) = 32896$.
`trace 2147483647 grep hello README`	Theo dõi toàn bộ các system call ($2^{31} - 1$, bật tất cả 31 bit đầu).
`grep hello README`	Chạy lệnh trực tiếp từ shell mà không qua `trace`. Xác nhận không có dòng vết nào bị rò rỉ (tính cô lập).
`trace 0 grep hello README`	Trường hợp biên với mặt nạ bằng 0. Lệnh vẫn chạy bình thường nhưng kernel không xuất bất kỳ log theo dõi nào.
`trace 2 usertests forkforkfork`	Theo dõi `SYS_fork` (mặt nạ $1 \ll 1 = 2$) trên tiến trình sinh nhiều thế hệ con cháu để kiểm tra tính kế thừa.
`trace 32896 grep hello no_file`	Cố tình truy xuất tệp không tồn tại để kiểm tra khả năng bắt lỗi và xuất thông báo từ hệ thống.
`trace abc grep hello README`	Kiểm tra xử lý lỗi người dùng khi nhập mặt nạ không phải chữ số. Chương trình từ chối thực thi và in hướng dẫn.

Table 1: Các kịch bản kiểm thử tính năng theo dõi system call

Kết quả thực nghiệm trên môi trường giả lập QEMU (Hình 1 và Hình 2) cho thấy các kịch bản kiểm thử đều hoạt động đúng theo đặc tả của đề bài:

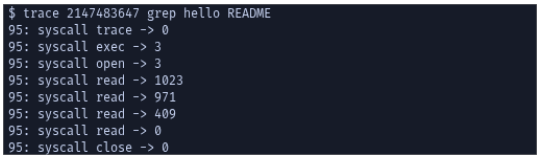
- **Theo dõi cơ bản và cô lập (Hình 1):** Hệ thống lọc chính xác từng system call theo bitmask (như `read` ở Hình 1a, `exec` và `open` ở Hình 1b, hoặc toàn bộ ở Hình 1c). Đồng thời, tiến trình shell không bị ảnh hưởng (Hình 1d và Hình 1e), bảo đảm tính cô lập.
- **Kế thừa đa thế hệ (Hình 2a):** Khi thực thi bài kiểm thử `forkforkfork`, các tiến trình con cháu liên tục kế thừa việc theo dõi `SYS_fork` với các PID tăng dần $(146, 147, \dots)$ cho đến khi bảng tiến trình đầy và trả về mã `-1`, kết thúc với `ALL TESTS PASSED`.
- **Xử lý ngoại lệ và tham số lỗi (Hình 2):** Khi gọi tệp không tồn tại (Hình 2b) hoặc truyền mặt nạ sai định dạng (Hình 2c), chương trình bắt lỗi chính xác và thông báo ra màn hình mà không làm sập hệ điều hành.



(a) Theo dõi `SYS_read` (mask = 32)



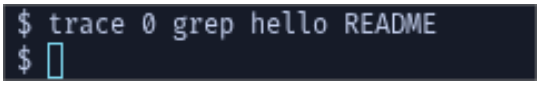
(b) Theo dõi `exec` và `open` (mask = 32896)



(c) Theo dõi toàn bộ syscall (mask = 2147483647)

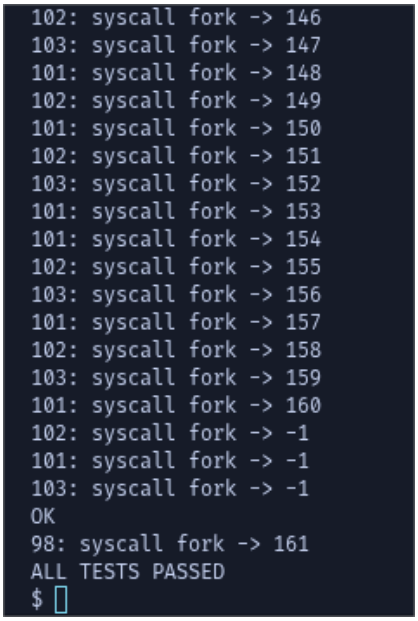


(d) Chạy trực tiếp ngoài shell (không qua trace)



(e) Mặt nạ rỗng (mask = 0)

Figure 1: Ảnh chụp kết quả kiểm thử các kịch bản theo dõi cơ bản và tính cô lập



(a) Kế thừa qua phân nhánh (`forkforkfork`)



(b) Truy xuất tệp không tồn tại



(c) Nhập sai cú pháp tham số (`mask = abc`)

Figure 2: Ảnh chụp kết quả kiểm thử tính kế thừa tiến trình và xử lý ngoại lệ

2 sysinfo.c

Để thu thập thông tin về tài nguyên hệ thống đang hoạt động, nhóm thiết lập thêm một system call mới với mã định danh `SYS_sysinfo = 24` trong file `kernel/syscall.h` [1]. Lời gọi hệ thống này nhận vào một đối số duy nhất là con trỏ trỏ tới cấu trúc `struct sysinfo` được định nghĩa trong file `kernel/sysinfo.h`, bao gồm 2 trường dữ liệu chính: `freemem` (tổng dung lượng bộ nhớ vật lý còn trống tính theo byte) và `nproc` (số lượng tiến trình có trạng thái khác `UNUSED`). Kernel có nhiệm vụ truy xuất các cấu trúc dữ liệu nội bộ để tính toán chính xác hai thông số này và sao chép an toàn về không gian người dùng.

2.1 Triển khai

Để thực hiện việc thu thập dữ liệu nhất quán và hiệu quả, nhóm chia tiến trình xử lý thành ba thành phần: điều phối tiếp nhận tại `sysproc.c`, đo lường bộ nhớ tự do trong `kalloc.c`, và đếm số lượng tiến trình còn sống trong `proc.c`. Đầu tiên, khi chương trình người dùng phát động lời gọi `sysinfo(&info)`, kernel tiếp nhận yêu cầu tại hàm `sys_sysinfo(void)` nằm trong `kernel/sysproc.c`. Tại đây, kernel sử dụng hàm trợ giúp `argaddr(0, &uaddr)` để trích xuất địa chỉ ảo của vùng nhớ đệm do user cung cấp. Sau đó, hàm lần lượt gọi hai hàm chuyên trách là `count_freemem()` và `count_proc()` để điền dữ liệu vào biến tạm `info` cục bộ trên Kernel Stack.

Quá trình tính toán lượng bộ nhớ trống được hiện thực trong hàm `count_freemem()` tại `kernel/kalloc.c`. Trong hệ điều hành xv6, toàn bộ bộ nhớ vật lý sau điểm kết thúc của kernel (`end`) đến giới hạn `PHYSTOP` (128 MB) được quản lý dưới dạng một danh sách liên kết đơn các trang nhớ tự do `kmem.freelist`. Mỗi phần tử trong danh sách này tương ứng với một khung trang (Page Frame) có kích thước cố định `PGSIZE = 4096` byte. Do đó, hàm `count_freemem()` duyệt qua toàn bộ danh sách liên kết để đếm số lượng trang trống khả dụng, sau đó nhân với `PGSIZE` để quy đổi ra tổng số byte còn rảnh. Vì xv6 hoạt động trên môi trường đa lõi (SMP), các CPU khác có thể đang đồng thời cấp phát (`kalloc`) hoặc giải phóng (`kfree`) các trang nhớ làm thay đổi liên kết của `freelist`. Vì vậy, hàm bắt buộc phải giữ khóa spinlock bằng lệnh `acquire(&kmem.lock)` trong suốt quá trình duyệt và nhả khóa `release(&kmem.lock)` ngay sau khi hoàn tất để bảo đảm tính toàn vẹn của danh sách.

Tương tự, quá trình đếm số tiến trình đang hoạt động được thực hiện bởi hàm `count_proc()` tại `kernel/proc.c`. Hệ thống quản lý toàn bộ các tiến trình thông qua một mảng tĩnh cố định `proc[NPROC]` gồm 64 phần tử. Một tiến trình được coi là đang hoạt động nếu trường trạng thái

của nó khác `UNUSED` (tức là đang ở các trạng thái `USED`, `SLEEPING`, `RUNNABLE`, `RUNNING` hoặc `ZOMBIE`). Để tránh tình trạng đọc phải dữ liệu rác hoặc xảy ra xung đột bộ nhớ khi một CPU khác đang thực thi phân nhánh (`fork`) hoặc thu dọn tiến trình (`exit`), hàm thực hiện duyệt tuần tự qua từng phần tử, chiếm giữ khóa spinlock `acquire(&p->lock)`, kiểm tra điều kiện trạng thái để tăng biến đếm và nhả khóa `release(&p->lock)` một cách an toàn.

Sao chép an toàn ra User Space : Sau khi đã thu thập đầy đủ thông tin vào biến cục bộ `struct sysinfo info`, bước quyết định là đưa kết quả ra vùng nhớ của chương trình người dùng. Vì kernel và user process hoạt động trên hai không gian địa chỉ ảo hoàn toàn biệt lập thông qua hai bảng phân trang riêng biệt (`kernel_pagetable` và `p->pagetable`), địa chỉ ảo `uaddr` của user không hề có ánh xạ hợp lệ trong bảng phân trang của kernel. Do đó, ta không thể dùng phép gán con trỏ C thông thường mà bắt buộc phải sử dụng hàm `copyout()` của xv6. Hàm này sẽ tự động phân giải cây phân trang của tiến trình gọi (`myproc()->pagetable`) để ghi dữ liệu trực tiếp vào đúng khung trang vật lý tương ứng. Nếu con trỏ `uaddr` bị lỗi (chẳng hạn con trỏ rỗng hoặc địa chỉ rác chưa được ánh xạ), hàm `copyout` sẽ trả về mã âm; khi đó `sys_sysinfo` lập tức từ chối và trả về -1 để ngăn ngừa lỗi sập hệ thống, ngược lại trả về 0 biểu thị thành công.

2.2 Kiểm thử

Sau khi hoàn tất quá trình triển khai, nhóm tiến hành đánh giá giải pháp thông qua bộ kiểm thử tích hợp `user/sysinfotest.c`. Bộ kiểm thử này được thiết kế để kiểm tra tính chính xác của số lượng trang nhớ trống và tiến trình hoạt động, cũng như khả năng xử lý an toàn khi người dùng truyền con trỏ không hợp lệ (Bảng 2).

Kết quả thực nghiệm trên môi trường giả lập QEMU (Hình 3) cho thấy chương trình đã vượt qua tất cả các bài kiểm tra:

- **Đo lường bộ nhớ chính xác và an toàn con trỏ (Hình 3):** Lời gọi hệ thống phản ánh đúng từng byte bộ nhớ vật lý tự do trong suốt quá trình cấp phát và thu hồi của `sbrk()`, đồng thời hàm `copyout()` phát hiện và xử lý an toàn khi người dùng truyền vào các địa chỉ ảo không hợp lệ mà không gây lỗi trang (page fault) cho kernel.
- **Cập nhật số lượng tiến trình:** Bộ đếm `nproc` phản ánh đúng sự thay đổi số lượng tiến trình khi tiến trình con được tạo và kết thúc; việc sử dụng spinlock `p->lock` giúp đồng bộ hóa quá trình đọc trạng thái của từng tiến trình trong bảng `proc`.

Kịch bản kiểm thử	**Mục đích kiểm tra**
`testcall()`	Kiểm tra gọi `sysinfo` với vùng đệm hợp lệ, đồng thời truyền con trỏ rác (`0xeaeb0b5b00002f5e`) để xác nhận kernel chặn lỗi thông qua `copyout` và trả về -1.
`testmem()`	Dùng hàm `sbrk()` cấp phát liên tục từng trang cho đến khi cạn kiệt RAM để kiểm tra `freemem == 0`; sau đó giải phóng và cấp phát lại để kiểm tra bộ đếm byte tăng giảm chuẩn xác.
`testproc()`	Ghi nhận `nproc` ban đầu, sinh tiến trình con bằng `fork()` để xác nhận `nproc` tăng đúng 1 đơn vị; sau khi cha thu hồi con qua `wait()`, xác nhận `nproc` trở về giá trị cũ và mã thoát của con hợp lệ.

Table 2: Các kịch bản kiểm thử tính năng thu thập thông tin hệ thống



Figure 3: Ảnh chụp kết quả thực thi chương trình kiểm thử `sysinfotest` trên QEMU

3 pstree.c

Trong hệ điều hành xv6, mỗi tiến trình khi sinh ra đều được hệ thống quản lý thông qua cấu trúc `struct proc` nằm trong bảng tiến trình `proc[NPROC]` tại `kernel/proc.c` [2]. Tuy nhiên, hệ thống mặc định lại không cung cấp một công cụ hay lời gọi hệ thống nào để người dùng có thể xem được mối quan hệ phả hệ cha – con giữa các tiến trình đang cùng chạy. Để xây dựng được tính năng hiển thị cây tiến trình tương tự như lệnh `pstree` trong môi trường Linux, nhóm cần thiết lập thêm một system call mới với mã định danh `SYS_ptree = 23` trong tệp `kernel/syscall.h` [1].

System call này sẽ nhận vào hai đối số: một con trỏ vùng đệm `buf` trỏ tới mảng cấu trúc và một số nguyên `max` đại diện cho số lượng tiến trình tối đa mà người dùng muốn lấy. Để cả tầng kernel lẫn user đều có thể hiểu chung một kiểu dữ liệu, nhóm đã định nghĩa cấu trúc `struct ptreeinfo` trong tệp tiêu đề `kernel/ptree.h` bao gồm 5 thông tin cơ bản nhất của một tiến trình: `pid` (mã tiến trình), `ppid` (mã tiến trình cha), `state` (trạng thái), `memsize` (kích thước bộ nhớ user) và `name` (tên tiến trình).

3.1 Triển khai

Để triển khai hoàn chỉnh tính năng này, nhóm chia công việc thành hai phần: xây dựng system call `ptree` trong kernel để thu thập dữ liệu an toàn, và viết chương trình `pstree.c` ở không gian người dùng (`user/`) để nhận dữ liệu rồi dựng thành cây hiển thị ra màn hình.

Đầu tiên, khi người dùng gọi hàm `ptree(procs, NPROC)` ở user space, kernel sẽ tiếp nhận yêu cầu tại hàm `sys_ptree(void)` trong file `kernel/sysproc.c`. Tại đây, nhóm sử dụng hai hàm trợ giúp quen thuộc của xv6 là `argaddr(0, &uaddr)` để lấy địa chỉ bộ đệm ảo của user và `argint(1, &max)` để lấy số lượng phần tử tối đa, sau đó chuyển tiếp các tham số này vào hàm xử lý lõi `get_ptree(uaddr, max)` đặt tại `kernel/proc.c`.

Khi hiện thực hàm `get_ptree`, kernel cần kiểm tra tính hợp lệ của tham số và sử dụng khóa để đồng bộ hóa dữ liệu. Đầu tiên, kernel tiến hành kiểm tra tính hợp lệ của đối số: nếu người dùng truyền vào `max <= 0` hoặc con trỏ rỗng (`uaddr == 0`), hệ thống sẽ lập tức từ chối và trả về -1. Kế tiếp, kernel duyệt qua bảng tiến trình `proc[NPROC]`. Vì xv6 vận hành trên môi trường đa lõi (SMP), các tiến trình khác có thể thay đổi trạng thái liên tục. Do đó, hàm bắt buộc phải gọi `acquire(&p->lock)` để khóa tiến trình hiện hành trước khi đọc. Nếu tiến trình chưa được sử dụng (`p->state == UNUSED`), ta nhả khóa `release(&p->lock)` và bỏ qua để chỉ giữ lại các tiến trình thực sự còn sống.

Đối với mỗi tiến trình hợp lệ, kernel trích xuất các thông tin cơ bản bao gồm `pid`, `state`, kích thước bộ nhớ `p->sz` và tên tiến trình bằng `safestrcpy`. Điểm đặc biệt quan trọng cần lưu ý là khi lấy mã tiến trình cha (`ppid`): tiến trình đầu tiên của hệ thống (`initproc`) hoàn toàn không có cha, tức là con trỏ `p->parent == 0`. Nếu truy xuất trực tiếp `p->parent->pid` mà không kiểm tra thì sẽ xảy ra lỗi giải tham chiếu con trỏ rỗng khiến toàn bộ kernel bị sập. Do đó, hệ thống chỉ lấy `p->parent->pid` khi `p->parent` tồn tại, ngược lại sẽ gán mặc định bằng 0.

Sau khi đã nạp đầy đủ thông tin vào biến tạm `info`, kernel lập tức gọi `release(&p->lock)` trước khi tiến hành chuyển dữ liệu ra ngoài. Việc giải phóng spinlock trước khi gọi `copyout` giúp tránh giữ khóa trong thời gian dài khi truy cập bộ nhớ người dùng, giảm thiểu nguy cơ tắc nghẽn và xung đột khóa. Vì kernel và user sở hữu hai bảng phân trang hoàn toàn độc lập (`kernel_pagetable` và `p->pagetable`), địa chỉ ảo `uaddr` của user không hề tồn tại trong không gian của kernel. Do đó, ta tuyệt đối không thể ép kiểu con trỏ C để gán giá trị trực tiếp, mà bắt buộc phải sử dụng hàm `copyout()` của xv6 để kernel tự động tra cứu bảng phân trang của user và ghi dữ liệu an toàn vào đúng ô nhớ vật lý trong RAM. Vị trí ghi phần tử thứ `count` được tính toán theo độ dời byte: dst = uaddr + count × sizeof(struct $ptreeinfo$). Nếu hàm `copyout` gặp lỗi (chẳng hạn user truyền địa chỉ ảo ngoài phạm vi cấp phát), kernel sẽ lập tức dừng lại và trả về -1.

Tái tạo cây và In : Sau khi kernel đã hoàn tất việc đổ dữ liệu phẳng về bộ đệm, bước tiếp theo là chương trình `user/pstree.c` sẽ dựng lại cây phả hệ cha – con và in ra màn hình. Để tuân thủ nguyên tắc tách biệt giữa tầng nhân (cung cấp dữ liệu thô) và tầng người dùng (xử lý định dạng và hiển thị), nhóm lựa chọn thuật toán duyệt theo chiều sâu (DFS – Depth-First Search theo thứ tự Pre-order) bằng hàm đệ quy `print_tree(pid, depth, procs, n)`. Nhóm quyết định dùng DFS thay vì duyệt theo chiều rộng (BFS – Breadth-First Search) bởi vì trên giao diện dòng lệnh vốn chỉ có thể xuất văn bản tuần tự từ trên xuống, DFS cho phép in ngay tiến trình cha rồi đi sâu vào toàn bộ các nhánh con cháu của nó trước khi rẽ sang nhánh khác, giúp cấu trúc thụt lề thể hiện quan hệ cha – con một cách liền mạch và trực quan nhất. Ngược lại, nếu dùng BFS (duyệt theo mức), các tiến trình cùng độ sâu sẽ bị in dàn trải liên tiếp làm đứt gãy mạch phả hệ, đồng thời còn đòi hỏi phải cài đặt thêm cấu trúc hàng đợi (`Queue`) phức tạp vốn không có sẵn trong thư viện xv6. Hơn nữa, DFS đệ quy tận dụng trực tiếp ngăn xếp gọi hàm (Call Stack) giúp mã nguồn ngắn gọn và không cần cấp phát thêm cấu trúc hàng đợi phụ trợ.

Vì hàm `printf` tối giản của xv6 không hỗ trợ định dạng thụt lề tự động (`%*s`) như thư viện chuẩn của C, nhóm dùng vòng lặp `for` để in ra `depth * 2` khoảng trắng tương ứng với độ sâu của tiến trình trên cây, sau đó in các trường thông tin theo đúng cú pháp đề bài yêu cầu: `<pid> <name> state=<state> mem=<memsize>`. Kế tiếp, hàm sẽ duyệt tìm toàn bộ các tiến trình con có `ppid` bằng với `pid` hiện tại và gọi đệ quy tiếp với `depth + 1`. Ở hàm `main()`, chương trình tìm các tiến trình gốc không có cha trong danh sách (`has_parent == 0` hoặc `ppid == 0`, tiêu biểu là `init`) để phát động hàm đệ quy từ gốc với `depth = 0`. Nhờ đó, cây tiến trình được hiển thị rõ ràng, trực quan và đúng chuẩn quan hệ phân cấp.

3.2 Kiểm thử

Sau khi hoàn tất quá trình triển khai system call `ptree` trong kernel và tiện ích `pstree` ở user space, nhóm tiến hành thiết lập các kịch bản kiểm thử nhằm kiểm tra tính đúng đắn của giải pháp từ cấu trúc hiển thị phả hệ, mức độ thụt lề cho đến việc xử lý tham số đầu vào (Bảng 3).

Kịch bản kiểm thử	**Mục đích kiểm tra**
`pstree` (Trực tiếp từ shell)	Kiểm tra hiển thị phả hệ cơ bản của hệ thống gồm tiến trình gốc `init`, tiến trình dòng lệnh `sh` và chính tiến trình `pstree`.
Cấu trúc thụt lề phân tầng	Xác nhận mỗi cấp độ sâu (`depth`) được thụt lề đúng 2 khoảng trắng, phản ánh chuẩn xác quan hệ cha – con phân nhánh từ trên xuống dưới.
Tính chính xác của trạng thái	Kiểm tra trường `state`: tiến trình đang thực thi xuất dữ liệu mang trạng thái `RUNNING` (4), trong khi các tiến trình cha chờ đợi mang trạng thái `SLEEPING` (2).
Xác thực thông tin bộ nhớ	Xác nhận trường `memsize` phản ánh đúng kích thước không gian nhớ của tiến trình (`p->sz`).
Xử lý tham số biên (Logic tầng nhân)	Đảm bảo hàm xử lý tại tầng kernel luôn kiểm tra điều kiện đầu vào, từ chối thực thi và trả về mã lỗi -1 an toàn khi người dùng truyền `max <= 0` hoặc con trỏ bộ đệm rỗng (`uaddr == 0`).

Table 3: Các kịch bản kiểm thử tính năng hiển thị cây tiến trình

Kết quả thực nghiệm trên môi trường giả lập QEMU (Hình 4) cho thấy chương trình hoạt động đúng theo đặc tả của bài thực hành:

- **Cấu trúc phả hệ phân tầng (Hình 4):** Cây tiến trình được dựng và in ra màn hình tuần tự với gốc là `init` (PID 1, depth = 0). Tiến trình con `sh` (PID 2, depth = 1) được thụt lề 2 khoảng trắng, và tiến trình cháu `pstree` (PID 5, depth = 2) do shell khởi sinh được thụt lề 4 khoảng trắng, thể hiện rõ ràng quan hệ phả hệ qua 3 thế hệ tiến trình.

- **Tính chính xác của dữ liệu và trạng thái:** Tất cả các thuộc tính bao gồm `pid`, tên tiến trình, trạng thái (`RUNNING` cho `pstree` và `SLEEPING` cho các tiến trình cha), cùng kích thước bộ nhớ (`mem=20480` và `mem=24576`) đều được kernel thu thập nhất quán và truyền an toàn ra không gian người dùng.

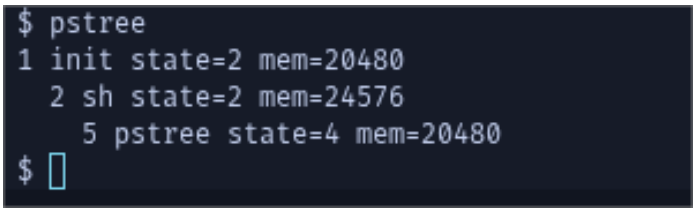


Figure 4: Ảnh chụp kết quả thực thi lệnh `pstree` trên môi trường QEMU

4 Bảng phân công nhiệm vụ

Bảng 4 tổng kết chi tiết phân công công việc, nhiệm vụ triển khai mã nguồn, xây dựng báo cáo và mức độ đóng góp của từng thành viên trong nhóm đối với Bài thực hành 02 – System Call.

Họ và tên	**MSSV & Vai trò**	**Nhiệm vụ đảm nhiệm**	**Đóng góp**
Nguyễn Phúc Khang	24120068 (Nhóm trưởng)	• Thiết kế và hiện thực system call `trace` (mã `SYS_trace = 22`, mặt nạ bitmask `trace_mask`). • Xử lý tính cô lập và kế thừa tiến trình con qua `allocproc()` và `fork()`. • Xây dựng chương trình người dùng `user/trace.c` và mảng điều phối `syscall_names[]`. • Soạn thảo nội dung báo cáo mục `trace.c`, tổng hợp tài liệu và quản trị kho Git.	100%
Hoàng Trọng Nghĩa	24120103 (Thành viên)	• Thiết kế và hiện thực system call `sysinfo` (mã `SYS_sysinfo = 24`, cấu trúc `struct sysinfo`). • Hiện thực hàm đếm bộ nhớ trống `count_freemem()` trong `kernel/kalloc.c` với khóa spin-lock `kmem.lock`. • Hiện thực hàm đếm tiến trình hoạt động `count_proc()` trong `kernel/proc.c`. • Triển khai cầu nối sao chép an toàn qua `copyout()`, kiểm thử `sysinfotest` và viết báo cáo mục `sysinfo.c`.	100%
Phạm Minh Trọng	24120151 (Thành viên)	• Thiết kế và hiện thực system call `ptree` (mã `SYS_ptree = 23`, cấu trúc `struct ptreeinfo`). • Hiện thực hàm `get_ptree()` trong kernel, xử lý con trỏ an toàn và cơ chế đồng bộ tránh deadlock. • Xây dựng chương trình người dùng `user/pstree.c` với thuật toán duyệt cây theo chiều sâu (DFS Pre-order). • Thiết kế các kịch bản kiểm thử phá hệ tiến trình, chụp minh chứng thực nghiệm và viết báo cáo mục `pstree.c`.	100%

Table 4: Bảng phân công nhiệm vụ và mức độ hoàn thành của các thành viên trong nhóm

References

[1] V.-L. Lê, *Project 2 – System call: Tài liệu hướng dẫn thực hành môn Hệ điều hành*, Khoa Công nghệ Thông tin, Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM, 2024, tài liệu thực hành khóa học Hệ điều hành (Moodle FIT-HCMUS).

[2] R. Cox, F. Kaashoek, and R. Morris, *xv6: A Simple, Unix-like Teaching Operating System*, 2024, rISC-V edition, revision 4. https://pdos.csail.mit.edu/6.828/2024/xv6/book-riscv-rev4.pdf.